



Máster en Inteligencia Artificial, Cloud Computing y DevOps Large Language Models

Guía de la entrega final

Alberto Ruiz-Arteaga González

1. Introducción

Esta guía describe la **evaluación final** de la asignatura **Large Language Models**. Tu trabajo consiste en crear un asistente turístico usando un LLM comercial (por ejemplo, OpenAI GPT-4o o Google Gemini) a través de su API.

El producto final será un **notebook** que reúna estos componentes:

- **RAG** sobre una guía turística que te facilitará el profesorado.
- **Diálogo multiturno** que mantenga el contexto de la conversación.
- Al menos una **llamada de función externa** que devuelva la predicción del tiempo (*get_weather(fecha)*), con gestión de errores básica.

Con este proyecto demostrarás tu dominio práctico de RAG, gestión de estado conversacional y *function calling* dentro de un flujo reproducible y bien documentado.

2. Objetivo general

Desarrollar un **prototipo conversacional** reproducible que combine búsqueda semántica, mantenimiento de contexto y llamadas a servicios externos, todo desde un único **notebook**.

3. Requisitos mínimos

1. Conexión con un LLM comercial

- Usa variables de entorno o un gestor seguro para la API key.
- Expón de forma clara los parámetros del modelo (temperatura, top-p, número máximo de tokens, etc.).

2. Implementación RAG

- Divide la guía en *chunks* y genera embeddings.
- Guarda el índice en un vector store (FAISS, Chroma u otro).
- Devuelve respuestas citando la fuente.

3. Diálogo multiturno

- Usa una memoria/contenedor de historial.
- Controla la longitud para no sobrepasar el límite de tokens.

4. Function call obligatoria

- Define un JSON Schema o usa Pydantic para definir, al menos, una función `get_weather``.
- Ejecuta la llamada (real o simulada) y maneja errores sencillos.
- Registra los intentos en un log.

4. Entregables

Elemento	Descripción
Repositorio	Estructura clara, README detallado, <i>requirements.txt</i> o <i>pyproject.toml</i> , <i>.gitignore</i> .
Notebook principal	Celdas ordenadas: carga de datos, indexación, conversación de ejemplos, pruebas, análisis.
Informe final	Diseño de la solución, decisiones técnicas, resultados, limitaciones y mejora futura.

5. Criterios de evaluación

Criterio	Peso
RAG y conexión con el LLM	30%
Diálogo multiturno y function calls	25%
Evaluación y análisis crítico	15%
Calidad del código y documentación	20%
Experiencia de usuario	10%
Bonus (despliegue, streaming, agentes, multimodalidad, observabilidad)	Hasta +20%

6. Conclusión

Con esta práctica aplicarás técnicas clave para construir asistentes basados en LLM: recuperación de información fiable, mantenimiento de contexto y uso de herramientas externas. El resultado mostrará tu capacidad para crear soluciones conversacionales reproducibles y bien evaluadas.

7. Anexo: criterios de evaluación detallados

7.1 RAG y conexión con el LLM (30 %)

- Estrategia de *chunking* y embeddings.
- Eficiencia del vector store y calidad de la recuperación.

7.2 Diálogo multiturno y function calls (25 %)

- Coherencia entre turnos.
- Definición y serialización correcta de la llamada.
- Al menos tres invocaciones correctas con manejo de fallos.

7.3 Evaluación y visualización (15 %)

- Conjunto reproducible de *prompts*.
- Presentación clara de métricas y gráficos.
- Discusión de casos límite y limitaciones.

7.4 Calidad del código y documentación (20 %)

- Cumplimiento de PEP 8 y docstrings claros.
- Código modular y reutilizable.
- Instrucciones de ejecución.

7.5 Experiencia de usuario (10 %)

- Interfaz intuitiva y mensajes de error útiles.
- Citaciones claras.
- Ejemplos que guíen al usuario.

Bonos técnicos (hasta +20 %)

Bonificación	Descripción
Despliegue web	Interfaz web (Streamlit, Gradio, etc.)
Streaming de tokens	Respuestas mostradas en tiempo real
Agentes o varias herramientas	Coordinación de varias herramientas mediante agentes
Multimodalidad	Soporte de imágenes o audio

